

Wi-Fi CSI Human Activity Recognition: Data Preparation, LSTM and BiLSTM

This documents the data and model workflow for the Wi-Fi Channel State Information (CSI) human activity recognition project. It explains the two datasets, the preprocessing scripts, the active training pipeline, and the LSTM/BiLSTM approaches.

Data-source note: The project report identifies the data as *Experiment 2* and *Experiment 3*, but does not provide an original dataset title, collection institution, public download URL, DOI, or license. Those details should be added when confirmed; they should not be guessed.

1. Project overview

The goal is to classify human activities from changes in Wi-Fi CSI measurements. Human movement affects the wireless channel through reflection, shadowing, and multipath propagation. A time-series model can learn patterns in these measurements and predict an activity for each short sequence of packets.

The combined task has eight classes:

- EMPTY — no person/activity present
- FALL — falling
- LYING — lying down
- SIT — sitting
- SIT-DOWN — transition from standing to sitting
- STAND — standing
- STAND-UP — transition from sitting to standing
- WALK — walking

SIT, STAND, and WALK are present in both selected experiments.

2. Dataset Source and Description

Original dataset and authors

The datasets used in this project originate from the research paper:

“Human Activity Recognition Using CSI Information with Nexmon”, published in *Applied Sciences* in 2021 by Jörg Schäfer, Baldev Raj Barrsiwal, Muyassar Kokhkhharova, Hannan Adil, and Jens Liebehenschel. The authors are affiliated with the Faculty of Computer Science and Engineering at Frankfurt University of Applied Sciences, Germany.

The paper investigates Wi-Fi-based Human Activity Recognition (HAR) using Channel State Information (CSI) extracted with the open-source Nexmon tool. The researchers collected CSI measurements using relatively inexpensive hardware, including Asus RT-AC86U

routers and Raspberry Pi devices, and evaluated machine-learning methods such as Long Short-Term Memory (LSTM) networks and Support Vector Machines (SVMs).

The original authors made the collected datasets publicly available. The official dataset record is available at <https://doi.org/10.21227/xr6j-0255>.

Publication reference:

Schäfer, J., Barrsiwal, B. R., Kokhkhharova, M., Adil, H., & Liebehenschel, J. (2021). *Human Activity Recognition Using CSI Information with Nexmon*. Applied Sciences, 11(19), 8860. <https://doi.org/10.3390/app11198860>

Selected experiments

This project combines data from two experiments described in the original paper:

Dataset	Original experiment	Activities
Experiment 2	Asus RT-AC86U, 80 MHz CSI measurements	FALL, SIT, SIT-DOWN, STAND, STAND-UP, WALK
Experiment 3	Asus RT-AC86U, 80 MHz CSI measurements	EMPTY, LYING, SIT, STAND, WALK

The original experiments use different activity subsets and different data-scaling approaches. Experiment 2 applies normalization, while Experiment 3 applies standardization in the original study. In this project, the selected recordings are combined to create a classification task with eight activity classes: EMPTY, FALL, LYING, SIT, SIT-DOWN, STAND, STAND-UP, and WALK.

CSI features

The input data consists of CSI measurements describing how Wi-Fi signals propagate through an indoor environment. Human movement changes the wireless channel, producing variations that can be learned by a neural network.

For this implementation, each packet is represented by 256 numeric CSI amplitude features. For a complex CSI value (H), its amplitude is calculated as:

$$|H| = \sqrt{(\operatorname{Re}(H))^2 + (\operatorname{Im}(H))^2}$$

The activity label is stored separately from the numerical input features and is used as the target for supervised classification.

The preprocessing and model-training procedures in this project are an implementation built on the selected datasets from the original paper. The windowing strategy, combined eight-class task, and current model configurations should be understood as part of this project's own pipeline rather than assumed to reproduce the original authors' complete experimental methodology.

Experiment 2: PCAP-to-CSV conversion (`convertCSI.py`)

`convertCSI.py` is a data-conversion utility for the raw Experiment 2 packet captures. It walks through activity folders, reads captures using `csiread.Nexmon` (configured with chip 4366c0 and bandwidth 80), calculates CSI amplitude, and writes a CSV for each recording.

CSI values are complex numbers. The amplitude is calculated for each packet and subcarrier as:

[$|H|$ =]

The output columns are named `SC_0` through `SC_255`, with a `Label` column containing the activity name inferred from the parent activity folder.

When to run it: only when starting from raw Experiment 2 .pcap files or when the Experiment 2 CSVs need to be regenerated. If the correct CSVs already exist, this conversion step is not required for every training run.

The script also applies a `MinMaxScaler` to each recording while converting it. If these converted files are used by the current pipeline, that per-recording scaling happens before the later shared `StandardScaler` in `dataset.py`. Confirm the provenance of the actual input CSVs before describing this as the exact scaling history of a particular experiment.

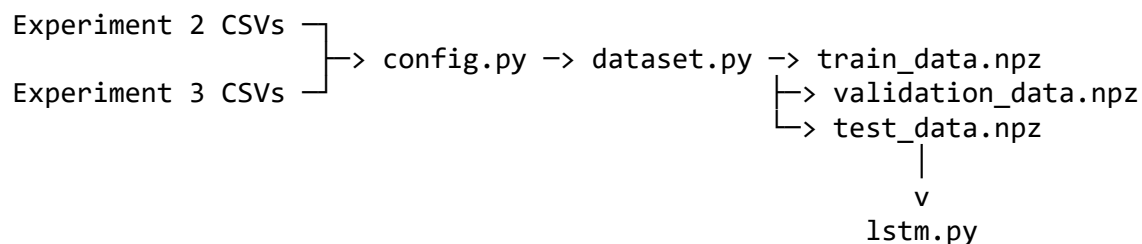
Experiment 3: CSV organization (`split.py`)

`split.py` prepares the original Experiment 3 CSV files. It infers the activity from the filename, normalizes activity names, assigns a `Label` column, and splits files into train and validation folders using a per-class split and random seed 42.

When to run it: when the original Experiment 3 CSVs need to be organized into the expected directory structure. If the folders are already prepared, do not rerun it without checking whether doing so would overwrite or change the current split.

3. Active preprocessing and dataset pipeline

The current model pipeline is:



The main files are:

- **`config.py`** — defines the dataset paths, class names, feature count, window parameters, random seed, and output locations.

- **dataset.py** — discovers and validates CSV recordings, determines labels, splits the records, scales the features, creates sliding windows, and saves the .npz datasets.
- **lstm.py** — loads the .npz datasets, trains the baseline LSTM, and evaluates it.
- **convertCSI.py and split.py** — opreparation steps, used only when the source data still needs conversion or organization.

Label handling

dataset.py first attempts to determine a label from the activity folder path. If a valid activity class can be inferred from the folder, that folder-derived label takes precedence over the CSV's Label column. The CSV label is a fallback when the path does not provide a recognized class.

For example, if a file is located in train/SIT/ but its CSV Label column says WALK, the active pipeline assigns SIT because the folder path identifies the class. Keep folder names, file contents, and labels consistent to avoid silently assigning an unexpected target.

The Label column and metadata columns are excluded from the 256 model input features. The preprocessing script checks that the selected feature columns are numeric, that there are exactly 256 of them, and that values are finite.

Train, validation, and test split

In the active pipeline, the existing validation folders are configured as the **test-source folders**. The records discovered in the configured training folders are split again into training and validation subsets, using a stratified split and random seed 42. Thus, the final validation set is created internally from the training sources; the existing validation folders are used as the final test set.

This is an important distinction from split.py, which creates an initial train/validation split for Experiment 3. The final split used by dataset.py depends on the folder paths configured in config.py.

Records are split before windows are generated. This helps prevent overlapping windows from the same recording from being distributed across different splits.

Scaling and window generation

dataset.py fits one shared StandardScaler on the training records only, then uses that fitted scaler to transform validation and test data. Fitting the shared scaler only on training data prevents the validation/test distributions from being used to estimate its parameters.

The script segments recordings into sliding windows with:

- **Window length:** 128 packets
- **Stride:** 64 packets
- **Maximum:** 2 windows per CSV recording
- **Features per packet:** 256

Each input sample therefore has shape (128, 256), and a batch has shape (batch_size, 128, 256). The stride allows adjacent windows to overlap, while the two-window limit reduces the influence of longer recordings.

The report lists the following prepared split sizes:

Split	Windows	Input shape
Training	2,934	(2934, 128, 256)
Validation	734	(734, 128, 256)
Test	772	(772, 128, 256)

These are the counts reported for the experiment. To verify that a current run reproduces them, inspect the generated .npz files and the manifests written by `dataset.py`.

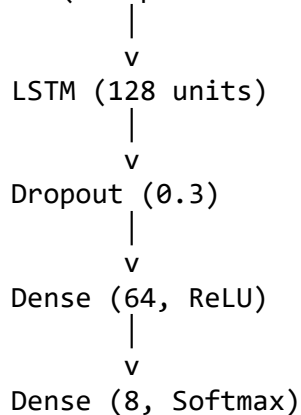
4. LSTM baseline

Why an LSTM?

A Long Short-Term Memory (LSTM) network is a recurrent neural network designed to learn patterns across sequential data. For CSI activity recognition, packet order matters because the signal changes over the course of an activity or transition.

Architecture implemented in `lstm.py`

Input window (128 packets × 256 features)



The final softmax layer produces a score/probability for each of the eight activity classes.

Training configuration

The supplied `lstm.py` configures:

- Optimizer: Adam
- Initial learning rate: 0.0005
- Loss: `sparse_categorical_crossentropy`
- Class weights to address class imbalance

- Early stopping based on validation loss
- ReduceLROnPlateau to lower the learning rate when validation loss stops improving
- ModelCheckpoint to save the best model

The script's actual settings should be treated as the source of truth when reproducing this implementation. Training settings in the report may differ from the current code version.

Reported baseline results

The project report gives the following LSTM results on a test set of 772 windows:

Metric	Reported result
Test accuracy	96.24%
Macro precision	95.70%
Macro recall	95.73%
Macro F1-score	95.65%
Weighted F1-score	96.23%
Correct predictions	743 / 772
Incorrect predictions	29

These are report-reported results, not a guarantee that every run will reproduce the same metrics. Results can vary with data versions, splits, software versions, and training conditions.

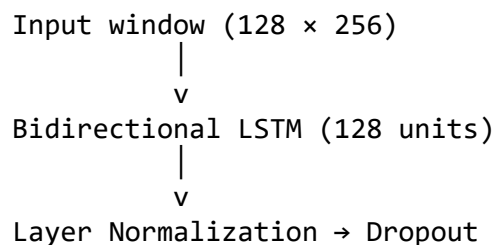
5. BiLSTM with attention

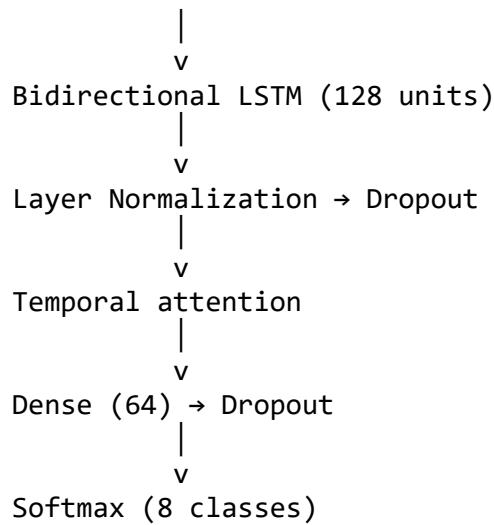
Why use a BiLSTM?

A bidirectional LSTM (BiLSTM) processes a sequence in both forward and backward directions. This lets the representation of a time step incorporate context from earlier and later packets within the same window. That can be useful for activities whose distinguishing patterns unfold over time, especially transitions such as SIT-DOWN and STAND-UP.

Reported architecture

The project report describes the improved model as two stacked bidirectional LSTM layers with 128 units each, followed by Layer Normalization, Dropout, temporal attention, a dense layer, and an eight-class softmax output:





Attention learns weights over time steps so that more informative parts of the sequence can contribute more strongly to the final prediction. Dropout helps regularize the model, and Layer Normalization helps stabilize intermediate activations.

Implementation note: the supplied Python files include the baseline `lstm.py`, but do not include a BiLSTM/attention implementation. The architecture and results in this section are therefore summarized from the project report, not verified against a supplied BiLSTM source file. Add the BiLSTM training script to the repository if readers should be able to reproduce that model.

Reported comparison

Metric	LSTM baseline	BiLSTM + attention
Test accuracy	96.24%	98.58%
Macro precision	95.70%	98.54%
Macro recall	95.73%	98.53%
Macro F1-score	95.65%	98.54%
Weighted F1-score	96.23%	98.57%
Correct predictions	743 / 772	761 / 772
Incorrect predictions	29	11

According to the report, test accuracy improved by 2.34 percentage points. The report also notes that EMPTY and FALL were classified perfectly in the test split, while errors remained among similar activities and transitions.

These results apply to the reported dataset and split. They do not establish that the same performance will hold for new people, rooms, hardware, or complete recordings. The report does not describe evaluation on entirely new environments and people.

6. Reproducing the active pipeline

1. Put the Experiment 2 and Experiment 3 CSVs into the directory structures configured in `config.py`.
2. If needed, use `convertCSI.py` to convert Experiment 2 .pcap files to CSV.
3. If needed, use `split.py` to organize/split the original Experiment 3 CSVs.
4. Check `config.py` so its paths match the local dataset folders.
5. Run `dataset.py` to create the prepared .npz datasets and preprocessing manifests.
6. Run `lstm.py` to train and evaluate the baseline LSTM.
7. Check the generated metrics, confusion matrix, training curves, saved model, and manifests.

The exact original data download location and license are not specified in the project report. Before publishing the dataset or providing download instructions, confirm the source and redistribution terms with the data provider.

7. Background references

The project report's background references include:

- S. Yousefi et al., "A Survey of Human Activity Recognition Using WiFi CSI," arXiv:1708.07129, 2017.
- M. Z. Khan et al., "Contactless Human Activity Recognition Using Deep Learning Methods Based on Wi-Fi CSI," 2023.
- S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.

These are background/method references. They should not be presented as the source of the project's raw data unless that relationship is independently confirmed.